WEEK 5 CHEAT SHEET

Lists, Tuples, Exceptions, Iteration & Assignment Explanations

Quick theory first; detailed reasoning for the assignment questions is at the end.

1. Lists, tuples and exceptions

List: an ordered, mutable collection. Mutable means you can change it after creation.

```
a = [1, 2, 3]
a[0] = 10          # [10, 2, 3]
a.append(4)        # [10, 2, 3, 4]
```

Tuple: an ordered, immutable collection. Its item assignments are not allowed after creation.

```
t = (10, 20)
# t[0] = 5  -> TypeError
single_item = (5,)  # comma makes it a tuple
```

Sequence unpacking: assigning sequence elements to separate variables. The number of targets should match the number of values.

```
x, y = (10, 20)   # x = 10, y = 20
```

Exception: a runtime error/condition that interrupts normal execution. `try` runs risky code; `except` handles a matching exception.

```
try:
    result = 10 / 0
except ZeroDivisionError:
    result = 0
```

String split: `s.split()` splits on runs of whitespace; `s.split(' ')` splits on each literal space and can preserve empty tokens between repeated spaces or at the ends.

append: `my_list.append(x)` mutates the existing list by adding one item at its end.

2. Iteration and recursion

Iteration repeats using loops. **Recursion** is when a function calls itself. Recursion needs a base case and each call must move toward it.

Pattern 1 — iterate until success: repeat a check/action until the required condition becomes true. Check that the loop can actually terminate.

Pattern 2 — accumulate: initialize a result and update it for each item. Common accumulators are a sum, count, product, or output list.

```
total = 0
for x in [2, 4, 6]:
    total += x
# total = 12

result = []
for x in [1, 2, 3]:
    result.append(x * 2)
# result = [2, 4, 6]
```

List comprehension: compact way to construct a list, optionally filtering items.

```
squares = [x*x for x in [1, 2, 3]]
positive = [x for x in data if x > 0]
```

Prime-checking idea: a prime number is an integer greater than 1 with no positive divisors other than 1 and itself. A prime-product function should multiply only values the specification says to include.

3. Writing correct functions

A function's **specification** is what the client says it must do. Similar-looking code can behave differently because the programmer made different assumptions.

Clarify edge cases before coding: empty input; strings with repeated spaces; uppercase/lowercase; invalid element types; zero or negative values; duplicates; and what to return if no valid answer exists.

Debug by tracing: track variables line by line, check loop bounds and updates, and test normal cases plus boundary cases. Never assume code is correct just because it looks reasonable.

Important: `type(x) is int` excludes booleans, while `isinstance(x, int)` treats booleans as integers because `bool` is a subclass of `int`.

4. Assignment questions with explanations

Question 1 — first_word(s)

Question: The function uses `s.split(' ')` and returns element 0. It catches `IndexError` and returns an empty string. Which inputs differ from the client's expected behaviour?

```
def first_word(s: str) -> str:
    try:
        s = s.split(' ')
        return s[0]
    except IndexError:
        return ""
```

Accepted answers shown: `'First word'` and `'firstword'`.

Why 'First word'? The function returns `'First'`, keeping the capital F. The client expected different behaviour for this test, likely a normalized/lowercase first word based on the intended examples. The code never calls `lower()`.

Why 'firstword'? There is no space, so splitting produces one token: `['firstword']`. The function returns the entire string. If the client expects the first word to be separated by a space, this input reveals the assumption.

Why is the empty string subtle? `''.split(' ')` gives `['']`, not an empty list. Therefore `s[0]` returns `''` without raising `IndexError`. The exception handler does not help with this case.

Better defensive version:

```
def first_word(s: str) -> str:
    words = s.split()
    return words[0] if words else ""
```

This handles empty/whitespace-only strings, but preserves case. Add `.lower()` only if the client wants lowercase output.

Question 2 — max_positive_int(data)

Question: Return the maximum positive integer in a list. Examples shown: `max_positive_int([1, -2, 3])` returns 3; `max_positive_int(['one', 2, 3.14])` returns 2. Which implementation matches the client?

What the examples tell us: negative integers are not candidates; a string such as `'one'` is ignored; and the float `3.14` is not treated as an integer. Thus the function must filter candidates rather than call `max(data)` on the raw list.

Compare common implementations:

• `return max(data)` fails on mixed incomparable values and does not filter negatives or floats.

• Filtering with `item > 0` can raise `TypeError` when an item is a string.

• Filtering with `type(item) is int` removes strings and floats, but it still keeps negative integers unless it also checks `item > 0`.

• The empty-candidate case must be handled; otherwise `max([])` raises `ValueError`.

Implementation consistent with the visible examples, assuming no positive integer means return 0:

```
def max_positive_int(data: list) -> int:
    values = [x for x in data if type(x) is int and x > 0]
    return max(values) if values else 0
```

Important limitation: the screenshot did not show the accepted option for this question, so the exact quiz option cannot be confirmed. The implementation above follows the visible examples and states its empty-list assumption explicitly.

Question 3 — factors(n, data)

Question: Find all factors of n among the numbers in `data`. Example: `factors(8, [1, 2, 3])` returns `[1, 2]`.

Accepted answer shown: “None of these.”

Why the obvious version is unsafe: `[x for x in data if n % x == 0]` assumes every `x` is numeric and nonzero. If `x == 0`, then `n % x` causes `ZeroDivisionError`. A string can also cause a type error.

Why the other visible options are problematic:

• The option checking only `n % x == 0` does not guard against zero or non-numeric values.

• The option checking `type(x) == int and x != 0` is safer for zero and types, but may not match a specification that accepts floats or other numeric values; it also makes an assumption about allowed input types.

• The option using `type(x) in (int, float)` accepts floats, which may not make sense for integer factors, and must still guard against zero.

• The option using a broad `try/except` hides errors rather than clearly validating inputs.

Example robust version if factors must be nonzero integers:

```
def factors(n: int, data: list) -> list:
    result = []
    for x in data:
        if type(x) is int and x != 0 and n % x == 0:
            result.append(x)
    return result
```

For `n = 8` and `data = [1, 2, 3]`, it appends 1 and 2 because 8 % 1 = 0 and 8 % 2 = 0, but not 3 because 8 % 3 = 2.

Question 4 — applying the lecture ideas

The remaining Week 5 screenshots provided were lecture-topic lists rather than additional complete graded questions. The examinable ideas are: list mutability, tuples, unpacking, exceptions, recursion versus iteration, accumulating results, prime checking, list comprehensions, and testing assumptions.

Quick self-test:

1. What does `append()` do? — Mutates a list by adding an item at the end.

2. Why does recursion need a base case? — To stop recursive calls.

3. What is an accumulator? — A variable updated repeatedly to build a result.

4. Why clarify requirements? — To avoid implementing assumptions the client did not intend.

5. Why guard against zero in factors? — Modulo by zero raises an exception.